

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

1. Entity

Entity ทำหน้าที่เป็นตัวแทนข้อมูลและกฎพื้นฐานของแต่ละสิ่งในระบบ เช่น Product, User, Payment หรือ Order โดยเก็บข้อมูลที่เกี่ยวข้องกับวัตถุนั้น ๆ และจัดการพฤติกรรมที่เกี่ยวข้องกับข้อมูลของตัวเอง

หลัก GRASP ที่ใช้คือ **Information Expert** เนื่องจาก Entity เป็นคลาสที่มีข้อมูลที่เป็นต่อการตัดสินใจหรือทำงานนั้น ๆ อยู่แล้ว จึงควรเป็นผู้รับผิดชอบพฤติกรรมที่เกี่ยวข้องกับข้อมูลของตัวเอง เช่น Product สามารถตรวจสอบจำนวนสินค้าในสต็อกได้

นอกจากนี้ยังช่วยให้เกิด **High Cohesion** เพราะ Entity รับผิดชอบเฉพาะข้อมูลและพฤติกรรมที่เกี่ยวข้องกับตัวเอง ไม่ควรรับผิดชอบงานติดต่อด้านข้อมูลหรือควบคุมกระบวนการทั้งหมดของระบบ

2. Repository

Repository ทำหน้าที่เป็นตัวกลางในการติดต่อกับฐานข้อมูล เช่น การเพิ่มข้อมูล ค้นหาข้อมูล แก้ไขข้อมูล และลบข้อมูล โดย Controller หรือ Service ไม่จำเป็นต้องเขียนคำสั่ง SQL โดยตรง

ส่วนนี้สอดคล้องกับหลัก **Indirection** เพราะ Repository ทำหน้าที่เป็นตัวกลางระหว่าง Service กับฐานข้อมูล ช่วยลดการพึ่งพาโดยตรงระหว่างสองส่วน

นอกจากนี้ยังช่วยให้เกิด **Low Coupling** เพราะหากมีการเปลี่ยนแปลงวิธีการจัดเก็บข้อมูล เช่น เปลี่ยนโครงสร้างฐานข้อมูลหรือเปลี่ยนระบบฐานข้อมูล ก็สามารถแก้ไขภายใน Repository ได้ โดยไม่จำเป็นต้องแก้ไข Service หรือ Controller ทั้งหมด

3. Service

Service ทำหน้าที่จัดการ Business Logic หรือกระบวนการทำงานหลักของระบบ เช่น การคำนวณราคา การตรวจสอบสต็อก การสร้างรายการชำระเงิน หรือการทำงานที่ต้องประสานข้อมูลจากหลาย Entity

การแยก Business Logic มาไว้ใน Service ช่วยให้เกิด **High Cohesion** เนื่องจาก Service รับผิดชอบเฉพาะกระบวนการทางธุรกิจ ไม่ต้องรับผิดชอบเรื่องการรับ Request หรือการติดต่อด้านข้อมูลโดยตรง

Service ยังช่วยสนับสนุน **Low Coupling** เพราะ Controller ไม่จำเป็นต้องรู้รายละเอียดว่ากระบวนการทางธุรกิจทำงานอย่างไร เพียงเรียกใช้ Service ที่ต้องการเท่านั้น

4. Controller

Controller เป็นจุดรับคำขอจากผู้ใช้งานหรือระบบภายนอก เช่น การกดปุ่มสั่งซื้อ การเพิ่มสินค้า หรือการชำระเงิน จากนั้น Controller จะส่งข้อมูลต่อไปยัง Service เพื่อดำเนินการ และส่งผลลัพธ์กลับไปยังผู้ใช้งาน

ส่วนนี้ใช้หลัก **Controller Pattern** โดยกำหนดให้ Controller เป็นผู้รับผิดชอบจัดการ Event หรือ Request จากภายนอกระบบ แต่ไม่ควรใส่ Business Logic จำนวนมากไว้ใน Controller

ตัวอย่างเช่น เมื่อผู้ใช้กดปุ่มยืนยันการชำระเงิน Controller จะรับข้อมูลรายการสินค้าแล้วเรียก PaymentService เพื่อดำเนินการชำระเงิน แทนที่จะตรวจสอบสต็อก คำนวณราคา และบันทึกฐานข้อมูลทั้งหมดภายใน Controller

1.2 อธิบาย High-Level SOLID Principles

1. SRP – Single Responsibility Principle

SRP คือ หลักการที่กำหนดให้แต่ละคลาสควรมีหน้าที่รับผิดชอบหลักเพียงอย่างเดียว หรือมีเหตุผลในการเปลี่ยนแปลงเพียงเรื่องเดียว

ในระบบมีการแบ่งหน้าที่ของคลาสออกอย่างชัดเจน เช่น

- **Controller** รับผิดชอบการรับ Request และส่ง Response
- **Service** รับผิดชอบ Business Logic
- **Repository** รับผิดชอบการติดต่อฐานข้อมูล
- **Entity** รับผิดชอบข้อมูลและพฤติกรรมของ Entity

2. OCP – Open/Closed Principle

OCP คือ คลาสควรเปิดให้สามารถขยายความสามารถได้ แต่ควรปิดไม่ให้ง่ายต่อการแก้ไขโค้ดเดิมโดยไม่จำเป็น

3. LSP – Liskov Substitution Principle

LSP คือ คลาสลูกที่สืบทอดจากคลาสแม่ควรสามารถนำมาใช้งานแทนคลาสแม่ได้โดยไม่ทำให้การทำงานของระบบผิดพลาด

4. ISP – Interface Segregation Principle

ISP คือ ไม่ควรบังคับให้คลาสต้อง Implement Method ที่ไม่ได้ใช้งาน ควรแบ่ง Interface ขนาดใหญ่ให้เป็น Interface ขนาดเล็กที่มีหน้าที่เฉพาะ

ตัวอย่างเช่น หากมี Interface ที่รวมความสามารถหลายอย่าง เช่น การชำระเงิน การคืนเงิน และการตรวจสอบสถานะการชำระเงิน แต่บางคลาสต้องการเพียงการชำระเงินเท่านั้น ก็ไม่ควรบังคับให้คลาสนั้นต้อง Implement ทุก Method

5. DIP – Dependency Inversion Principle

DIP คือ Module ระดับสูงไม่ควรขึ้นอยู่กับ Module ระดับต่ำโดยตรง แต่ควรขึ้นอยู่กับ Abstraction หรือ Interface

ในระบบสามารถประยุกต์ใช้ได้โดยให้ Service ติดต่อกับ Repository Interface แทนการผูกกับ Repository ที่มีรายละเอียดการทำงานของฐานข้อมูลโดยตรง

1.3 อธิบาย Strategy Pattern

ระบบใช้ Strategy Pattern เพื่อแยกวิธีการคำนวณส่วนลดแต่ละประเภทออกจากกัน ทำให้สามารถเลือกใช้ส่วนลดได้ตามเงื่อนไขโดยไม่ต้องแก้ไขโค้ดหลักของระบบ

- DiscountStrategy: Interface กำหนดมาตรฐานการคำนวณส่วนลด
- NoDiscountStrategy: ไม่มีส่วนลด (จ่ายราคาเต็ม)
- StudentDiscountStrategy: คำนวณส่วนลดสำหรับนักเรียน/นักศึกษา
- SeasonalSaleStrategy: คำนวณส่วนลดตามโปรโมชั่นตามฤดูกาล
- DiscountContext: เลือกและเรียกใช้ Strategy ที่เหมาะสมในการคำนวณราคา

ประโยชน์ด้าน Open/Closed Principle (OCP)

ระบบสามารถเพิ่มส่วนลดใหม่ เช่น MemberDiscountStrategy ได้โดยสร้างคลาสใหม่ที่ใช้ DiscountStrategy โดยไม่ต้องแก้ไข DiscountContext หรือโค้ดเดิม ทำให้ระบบขยายฟังก์ชันได้ง่าย ลดการเกิดข้อผิดพลาด และดูแลรักษาได้สะดวก.

1.4 อธิบาย Layered Architecture

Layered Architecture คือการแบ่งระบบออกเป็นชั้นตามหน้าที่ เพื่อให้แต่ละส่วนรับผิดชอบงานของตัวเอง โดยระบบแบ่งเป็น **Controller, Service และ Repository**

- **Controller Layer** รับ Request จากผู้ใช้ ตรวจสอบข้อมูลเบื้องต้น และส่งต่อให้ Service
- **Service Layer** จัดการ Business Logic เช่น การคำนวณราคา ตรวจสอบส่วนลด และประมวลผลคำสั่งซื้อ
- **Repository Layer** รับผิดชอบการติดต่อและจัดการข้อมูลในฐานข้อมูล

การแยก **Service Layer** ออกจาก Controller และ Repository ช่วยให้แต่ละส่วนมีหน้าที่ชัดเจน หากนำ Business Logic ไปไว้ใน Controller จะทำให้ Controller มีโค้ดมากและแก้ไขได้ยาก ส่วน Repository ควรเน้นเฉพาะการเข้าถึงข้อมูล ไม่ควรรับผิดชอบกฎทางธุรกิจ

ด้าน **Low Coupling** ช่วยลดการพึ่งพาระหว่างแต่ละ Layer

ทำให้สามารถแก้ไขหรือเปลี่ยนแปลงส่วนหนึ่งได้โดยไม่กระทบส่วนอื่นมากเกินไป ส่วน **High Cohesion** ช่วยให้แต่ละ Layer รวมเฉพาะงานที่เกี่ยวข้องกัน ทำให้โค้ดเป็นระเบียบ เข้าใจง่าย ทดสอบ และบำรุงรักษาได้ง่ายขึ้น.

1.5 อธิบาย Execution Flow

- **Browser** → **Controller** รับ HTTP Request และตรวจสอบข้อมูลเบื้องต้น
- **Controller** → **Service** ส่งข้อมูลไปยัง Service เพื่อประมวลผล Business Logic
- **Service** → **Repository** เรียก Repository เพื่อดึงหรือบันทึกข้อมูลใน PostgreSQL
- **Service** → **Strategy** หากเป็นการซื้อเกม ระบบจะเลือก DiscountStrategy ที่เหมาะสม เช่น NoDiscountStrategy, StudentDiscountStrategy หรือ SeasonalSaleStrategy เพื่อคำนวณส่วนลดและราคาสุทธิ
- **Repository** → **PostgreSQL** บันทึกหรือดึงข้อมูลตามที่ Service ร้องขอ
- **ผลลัพธ์** → **Controller** → **Browser** ส่งผลลัพธ์กลับไป Browser เพื่อแสดงผลข้อมูลให้ผู้ใช้

ส่วนที่ 2: Code Implementation & Explanation

1. Entity Layer — model/Game.java

เก็บโครงสร้างข้อมูลหลักของระบบ (เช่น ชื่อเกม, ราคา, ประเภท) มักจะ map กับตาราง Database ผ่าน JPA Annotation (@Entity, @Id เป็นต้น) ชั้นนี้ไม่มี Dependency Injection เพราะเป็นแค่ Data Model ไม่มี Business Logic

2. Repository Layer — repository/

ทำหน้าที่ติดต่อฐานข้อมูล โดยปกติจะ extend JpaRepository<Game, Long> ซึ่ง Spring Data JPA จะสร้าง Implementation ให้อัตโนมัติ ไม่ต้องเขียนเอง

3. Strategy Package — หัวใจของ Design Pattern นี้

strategy/

```
|—— DiscountStrategy.java      (Interface)
|—— NoDiscountStrategy.java     (Implementation)
|—— SeasonalSaleStrategy.java   (Implementation)
|—— StudentDiscountStrategy.java (Implementation)
|—— DiscountContext.java        (ตัวเลือก Strategy ที่จะใช้)
```

- **DiscountStrategy** เป็น Interface กำหนด method กลาง เช่น double applyDiscount(double price)
- **NoDiscountStrategy / SeasonalSaleStrategy / StudentDiscountStrategy** คือ Concrete Class ที่ implement interface ด้านบน แต่ละคลาสคำนวณส่วนลดต่างกันไปตามเงื่อนไข (เช่น ไม่ลดเลย, ลดตามเทศกาล, ลดสำหรับนักเรียน)
- **DiscountContext** ทำหน้าที่เป็นตัวกลาง (Context) ที่ถือ reference ไปยัง DiscountStrategy ที่เลือกใช้ แล้ว delegate การคำนวณไปให้ Strategy นั้น ๆ

จุดเด่นของ Pattern นี้คือสามารถสลับ Strategy ได้โดยไม่ต้องแก้ไขโค้ดเดิม (Open/Closed Principle)

4. Service Layer — service/GameService.java

รวม Business Logic หลัก เช่น ดึงข้อมูลเกมจาก Repository แล้วนำไปคำนวณราคาหลังหักส่วนลดผ่าน DiscountContext

5. Controller Layer — controller/GameController.java

รับ HTTP Request (@RestController, @GetMapping, @PostMapping ฯลฯ) แล้วเรียกใช้ Service เพื่อประมวลผล จากนั้นส่ง Response กลับเป็น JSON

Dependency Injection แบบ Constructor Injection ในทุก Layer

หลักการคือ แต่ละ Class จะรับ Dependency ผ่าน Constructor แทนการใช้ @Autowired บน field โดยตรง ซึ่งเป็นแนวทางที่ Spring แนะนำ เพราะ:

- ทำให้ Object ที่สร้างขึ้นมี Dependency ครบตั้งแต่แรก (Immutable, ไม่มี null field)
- ทดสอบง่ายกว่า (Unit Test สามารถ mock แล้วส่งเข้า constructor ได้ตรง ๆ โดยไม่ต้องพึ่ง Spring Context)
- ถ้าใช้ Lombok @RequiredArgsConstructor กับ field ที่เป็น final จะไม่ต้องเขียน constructor เองด้วยซ้ำ

ส่วนที่ 3: Web Application & Database Screenshots

1. การเพิ่มเกมลงในลิสต์รายการ

← กลับไปรายการเกม

+ เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

🔍 ชื่อเกม

เช่น The Legend of Zelda: Tears of the Kingdom

🎮 แนวเกม (Genre)

เช่น Action, RPG, Adventure

💻 แพลตฟอร์ม (Platform)

เช่น PC, PS5, Nintendo Switch

⭐ คะแนน (Rating)

0.0 — 10.0

📅 ราคาปกติ (บาท)

เช่น 1990

% ส่วนลด (Strategy)

ราคาปกติ (0%)

📅 วันวางจำหน่าย (Release Date)

วว/ดด/ปปปป

💾 บันทึก

✖ ยกเลิก

2. ผลลัพธ์หลังการเพิ่มรายการเกม

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380402-4 SEC 3)	Action RPG	PC / PS5	★ 9.8	25/02/2025	ส่วนลดดิจิทัล (10%)	1611.00 1790.00	
2	Ark	Action RPG	PC / PS5	★ 10.0	06/12/2025	ส่วนลดเทศกาล (20%)	14400.00 18000.00	

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3. หน้าต่างการแก้ไขรายการเกม อัปเดต (ส่วนลดจาก 10% เป็น 20%)

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Elden Ring (673380402-4 SEC 3)

ชื่อเกม

Elden Ring (673380402-4 SEC 3)

แนวเกม (Genre)

Action RPG

แพลตฟอร์ม (Platform)

PC / PS5

คะแนน (Rating)

9.8

ราคาปกติ (บาท)

1790.0

% ส่วนลด (Strategy)

ส่วนลดเทศกาล (20%)

วันวางจำหน่าย (Release Date)

วว/ดด/ปปปป

อัปเดต

ยกเลิก

4. ผลลัพธ์การอัปเดตรายการเกม

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+ เพิ่มเกมใหม่

✅ อัปเดตข้อมูลสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Ark	Action RPG	PC / PS5	★ 10.0	06/12/2025	ส่วนลดพิเศษ (20%)	14400.00 18000.00	 
2	Elden Ring (673380402-4 SEC 3)	Action RPG	PC / PS5	★ 9.8	06/12/2525	ส่วนลดพิเศษ (20%)	1432.00 1790.00	 

📖 ทั้งหมด 2 เกม

5. หน้าต่างการลบ

← กลับไปรายการเกม

ยืนยันการลบ



คุณแน่ใจหรือไม่?

การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกมArk

แนวเกมAction RPG

แพลตฟอร์มPC / PS5

คะแนน★ 10.0

ราคา18000.00 บาท

 ยืนยันลบ

← ยกเลิก

6. ผลลัพธ์การลบรายการเกม

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+ เพิ่มเกมใหม่

✓ ลบข้อมูลสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380402-4 SEC 3)	Action RPG	PC / PS5	★ 9.8	06/12/2525	ส่วนลดเทศกาล (20%)	1432.00 1799.00	 

ทั้งหมด 1 เกม

7. หน้า database (pgAdmin4)

